Python Scripting

NCSC101

Prof. Pranay Kumar Saha

October 8, 2025

What is the os Module?

- A built-in Python module that provides a portable way of using operating system-dependent functionality.
- It allows your Python scripts to interact with the OS, performing tasks like:
  - Navigating the file system
  - Creating, moving, and deleting files and directories
  - Accessing environment variables
  - Running system commands
- **Key Idea:** It abstracts away OS differences (e.g., path separators `/` vs `\`).

Importing the os Module

- Importing the **os** module is simple.
- You just need one line at the top of your script.

```
# Import the os module
import os

# Now you can use its functions,
# e.g., os.getcwd()
```

- We will also frequently use the **os.path** submodule, which is imported automatically with **os**.

`os.getcwd()`: Get Current Directory

- The **`os.getcwd()`** function returns the **C**urrent **W**orking **D**irectory (CWD) as a string.
- This is the folder where your Python script is ”living” and where it will look for files by default.

os.getcwd(): Get Current Directory

```python
import os

# Get the current working directory
cwd = os.getcwd()

print(f"Current Directory: {cwd}")

# On Windows, might print:
# C:\Users\YourUser\MyProject

# On Linux/macOS, might print:
# /home/YourUser/MyProject
```

os.chdir(): Change Directory

- The **os.chdir()** function changes the CWD to a new path.

```
import os
print(f"Old CWD: {os.getcwd()}")
# Change directory
try:
    os.chdir("/home/YourUser/Documents")
    print(f"New CWD: {os.getcwd()}")
except FileNotFoundError:
    print("Directory not found!")
except NotADirectoryError:
    print("That's a file, not a directory!")
except PermissionError:
    print("You don't have permission!")
```

`os.listdir()`: List Directory Contents

- **`os.listdir()`** returns a list of strings containing the names of the entries (files and directories) in a given path.
- If no path is specified, it lists the contents of the CWD.

os.listdir(): List Directory Contents

```
import os
# List contents of the current directory
contents = os.listdir()
print(contents)
# ['file1.txt', 'my_folder', 'script.py']
# List contents of a specific directory
try:
    other_contents = os.listdir("/home/YourUser")
    print(other_contents)
except FileNotFoundError:
    print("Path does not exist!")
```

os.mkdir(): Make a Single Directory

- **os.mkdir()** creates a new directory at the specified path.
- It can only create a single directory at a time.
- It will raise a **FileExistsError** if the directory already exists.
- It will raise a **FileNotFoundError** if a parent directory in the path doesn't exist.

```
import os
try:
    os.mkdir("new_folder")
    print("Folder 'new_folder' created!")
except FileExistsError:
    print("Folder 'new_folder' already exists.")
```

`os.makedirs()`: Make Multiple Directories

- **`os.makedirs()`** is like **`mkdir()`**, but it can create all the intermediate "parent" directories in a path.
- This is extremely useful for creating nested folder structures.
- By default, it also raises **`FileExistsError`** if the final directory exists.

os.makedirs(): Make Multiple Directories

```python
import os

# This will create 'parent', 'child', and 'grandchild'
try:
    os.makedirs("parent/child/grandchild")
    print("Nested directories created!")
except FileExistsError:
    print("Directory structure already exists.")

# Common pattern:
os.makedirs("path/to/my/data", exist_ok=True)
# exist_ok=True prevents the error
```

os.rmdir(): Remove a Single Directory

- **os.rmdir()** removes a single directory.
- **CRITICAL:** The directory **must be empty**.
- If the directory is not empty, it will raise an **OSError**.
- If the path doesn't exist, it raises **FileNotFoundError**.

```
import os

try:
    os.rmdir("empty_folder")
    print("Folder 'empty_folder' removed.")
except FileNotFoundError:
    print("Folder not found.")
except OSError:
    print("Folder is not empty!")
```

os.removedirs(): Remove Multiple Directories

- **os.removedirs()** is the recursive counterpart to **makedirs()**.
- It attempts to remove all directories in a path, starting from the innermost one.
- It stops as soon as it encounters a non-empty directory.

```
import os
# Assume you have a structure:
# parent/child/grandchild (all
    empty)
try:
    # This will remove all three
    directories
    os.removedirs("parent/child/
    grandchild")
    print("Removed nested
    directory structure.")
except OSError:
    print("Could not remove
    directories.")
    print("Are they all empty?")
```

`os.rename()`: Rename a File or Directory

- **`os.rename()`** renames a file or directory.
- It can also be used to **move** a file or directory by providing a new path in the destination.

os.rename(): Rename a File or Directory

```python
import os
# --- Rename a file ---
try:
    os.rename("old_name.txt", "new_name.txt")
except FileNotFoundError:
    print("File not found.")
# --- Move a file ---
# (This moves 'file.txt' into 'my_folder')
try:
    # Ensure 'my_folder' exists first!
    os.makedirs("my_folder", exist_ok=True)
    os.rename("file.txt", "my_folder/file.txt")
except OSError as e:
    print(f"Error moving file: {e}")
```

`os.remove()` / `os.unlink()`

- **`os.remove()`** is used to delete a **file**.
- **`os.unlink()`** is a synonym for **`os.remove()`**.
- You cannot use this to delete a directory (use **`os.rmdir()`** for that).

os.remove() / os.unlink()

```python
import os

# Let's create a file first
with open("file_to_delete.txt", "w") as f:
    f.write("I am temporary.")

# Now delete it
try:
    os.remove("file_to_delete.txt")
    print("File deleted.")
except FileNotFoundError:
    print("File was already gone.")
except IsADirectoryError:
    print("That's a directory, not a file!")
```

`os.stat()`: Get File Metadata

- **`os.stat()`** returns a rich object containing metadata about a file or directory.
- This includes:
  - **`st_size`**: Size in bytes
  - **`st_mtime`**: Last modification time (as a timestamp)
  - **`st_ctime`**: Creation time (on Windows)
  - **`st_atime`**: Last access time

os.stat(): Get File Metadata

```
import os
import datetime
try:
    stat_info = os.stat("my_file.txt")

    print(f"File size: {stat_info.st_size} bytes")

    mod_time = datetime.datetime.fromtimestamp(
        stat_info.st_mtime
    )
    print(f"Last modified: {mod_time}")

except FileNotFoundError:
    print("File not found.")
```

Introduction to `os.path`

- The **`os`** module has a very important submodule: **`os.path`**.
- This module contains functions for **manipulating file paths** in a cross-platform way.
- This is the key to writing code that works on Windows, macOS, and Linux without changes.
- It is available automatically when you **`import os`**.

`os.path.join()`: The Right Way to Build Paths

- **Never** build paths by adding strings like **`"folder" + "/" + "file.txt"`**!
- This breaks on different operating systems (Windows uses **`\`**, Linux/macOS use **`/`**).
- **`os.path.join()`** intelligently joins path components using the correct separator for the current OS.

os.path.join(): The Right Way to Build Paths

```python
import os

# Create a path in a cross-platform way
path = os.path.join("my_folder", "data", "file.txt")

print(path)

# On Windows, prints:
# my_folder\data\file.txt

# On Linux/macOS, prints:
# my_folder/data/file.txt
```

`os.path.basename()` & `os.path.dirname()`

- These functions intelligently split a path into its components.
- **`os.path.basename()`**: Returns the final component of a path (the file or directory name).
- **`os.path.dirname()`**: Returns the path to the directory containing the final component.

os.path.basename() & os.path.dirname()

```
import os
path = "/home/user/project/data.csv"
file_name = os.path.basename(path)
print(f"File Name: {file_name}")
# Prints: data.csv

dir_name = os.path.dirname(path)
print(f"Directory: {dir_name}")
# Prints: /home/user/project
```

os.path.split()

- **os.path.split()** does the same job as **basename** and **dirname**, but returns both parts as a tuple.

```
import os

path = "/home/user/project/data.csv"

# Splits path into (head, tail)
parts = os.path.split(path)

print(parts)
# Prints: ('/home/user/project', 'data.csv')

print(f"Head: {parts[0]}")
print(f"Tail: {parts[1]}")
```

os.path.exists(): Check if Path Exists

- **os.path.exists()** returns **True** if a path exists.
- It returns **True** for both files and directories.
- This is one of the most commonly used functions for validation.

```
import os

path = "my_folder/my_file.txt"

if os.path.exists(path):
    print(f"'{path}' exists!")
    # Now we can safely open it
    with open(path, "r") as f:
        print(f.read())
else:
    print(f"'{path}' does not exist!")
```

os.path.isfile() & os.path.isdir()

- After confirming a path exists, you often need to know *what* it is.
- **os.path.isfile()**: Returns **True** if the path is an existing **file**.
- **os.path.isdir()**: Returns **True** if the path is an existing **directory**.

```
import os

path = "my_folder"

if os.path.exists(path):
    if os.path.isfile(path):
        print(f"'{path}' is a file.")
    elif os.path.isdir(path):
        print(f"'{path}' is a directory.")
else:
    print(f"'{path}' does not exist.")
```

Example: Walking a Directory Tree

- A common task is to “walk” through a directory and all its subdirectories.
- **`os.walk()`** is a generator that does this for you.
- For each directory, it yields a 3-tuple: **`(dirpath, dirnames, filenames)`**

Example: Walking a Directory Tree

```python
import os

# Walk the current directory
for dirpath, dirnames, filenames in os.walk("."):
    print(f"\nCurrently in: {dirpath}")

    print(f"Subdirectories: {dirnames}")

    print(f"Files: {filenames}")

    # Example: Find all .py files
    for f in filenames:
        if f.endswith(".py"):
            print(f"Found Python file: {os.path.join(dirpath, f)
    }")
```

os.name: Checking the OS

- **os.name** returns the name of the operating system-dependent module.
- This lets you write code that runs differently on different platforms.
- The main values are:
  - **'nt'**: Windows
  - **'posix'**: Linux, macOS, and other Unix-likes

os.name: Checking the OS

```
import os
if os.name == 'nt':
   print("Running on Windows!")
   # Windows-specific command
   clear_command = "cls"
elif os.name == 'posix':
   print("Running on Linux or macOS!")
   # POSIX-specific command
   clear_command = "clear"
else:
   print(f"Running on unknown OS: {os.name}")
   clear_command = None
```

os.system(): Running Shell Commands

- Executes a command in a subshell.
- Returns the exit code of the command.

```
import os

# List files (Linux/macOS)
if os.name == 'posix':
   os.system("ls -l")

# List files (Windows)
if os.name == 'nt':
   os.system("dir")
```

Summary

- The **os** module is your bridge to the operating system.
- **Directories:** `getcwd()`, `chdir()`, `listdir()`, `mkdir()`, `makedirs()`, `rmdir()`, `removedirs()`.
- **Files:** `rename()`, `remove()`, `stat()`.
- **`os.path`:** Your best friend for cross-platform path logic.
  - **Always use `os.path.join()`!**
  - Use `exists()`, `isfile()`, `isdir()` for validation.
  - Use `basename()`, `dirname()`, `split()` to dissect paths.
- **System:** Access `os.environ` for variables, and use **`subprocess`** (not `os.system`) for running commands safely.

Python Scripting

Thank You for Listening!